AizoAutoGrading projekta tetsēšana

Roberts Treize, Alberts Druska

1. Funkcionālie un nefunkcionālie testi

Dokuments pieejams šeit: Google Docs dokuments

1.1. Funkcionālie testi (6)

- Vairākas atbildes vienam jautājumam → rezultāts `None`
- Pareizi aizpildīta testa lapa → pareizas atbildes un kods
- Daļēji iekrāsotas atbildes → korekta atpazīšana
- Dzēstas atbildes → netiek ņemtas vērā
- Trūkstošas atbildes → tukšas vērtības
- Nekorekti aizpildīta lapa → kļūdu apstrāde

1.2. Nefunkcionālie testi (6)

- Veiktspēja (apstrādes laiks $\sim$ 1.33 s)
- Atmiņas patēriņš ($\geq$ 450 MB)
- Stabilitāte dažādās OS (Ubuntu, Windows)
- Ierīces noslodze (telefona pārkaršana, baterija)
- Vienlaicīgi lietotāji (multi-user piekļuve)
- Drošība (kameras piekļuve tīklā)

2. Testēšanas metodoloģija

Tika izmantotas divas testēšanas pieejas:

1. **Vienības testēšana** — pārbauda atsevišķu funkciju uzvedību izolēti.
2. **Integrācijas testēšana** — pārbauda vairāku sistēmas daļu kopīgu darbību.

Testēšanai izmantoti:

- `pytest`;
- `unittest.mock.patch`;
- sintētiski ģenerēti attēli ar `NumPy` un `OpenCV`;
- Flask testēšanas klients HTTP pieprasījumu pārbaudei.

3. Vienības testi

3.1. 1. tests: tukšas studenta atbildes

Testētā problēma: pārbauda, vai sistēma spēj korekti normalizēt situāciju, kad students nav aizpildījis nevienu atbildi.

Ievade:

```
answer_sheet = {"1": "A", "2": "B", "3": "C"}
student_sheet = {}
```

Koda fragments:

```
result = dataNormalize(student_sheet, answer_sheet)
assert result == {"1": None, "2": None, "3": None}
```

Sagaidāmais rezultāts: visiem jautājumiem tiek piešķirta vērtība `None`.

Faktiskais rezultāts: tests izpildīts veiksmīgi, rezultāts sakrita ar sagaidīto.

3.2. 2. tests: tukša atbilžu lapa

Testētā problēma: pārbauda, kā funkcija uzvedas, ja nav pieejama neviena pareizā atbilde.

Ievade:

```
student_sheet = {"1": "A", "2": "B", "3": "C"}
answer_sheet = {}
```

Koda fragments:

```
result = dataNormalize(student_sheet, answer_sheet)
assert result == {}
```

Sagaidāmais rezultāts: tiek atgriezta tukša vārdnīca.

Faktiskais rezultāts: tests izpildīts veiksmīgi.

3.3. 3. tests: trūkstošas studenta atbildes

Testētā problēma: pārbauda, vai trūkstošie jautājumi tiek aizpildīti ar `None`.

Ievade:

```
answer_sheet = {"1": "A", "2": "B", "3": "C", "4": "D"}
student_sheet = {"1": "A", "3": "C"}
```

Koda fragments:

```
result = dataNormalize(student_sheet, answer_sheet)
assert result == {"1": "A", "2": None, "3": "C", "4": None}
```

Sagaidāmais rezultāts: 2. un 4. jautājums tiek piepildīti ar `None`.

Faktiskais rezultāts: tests izpildīts veiksmīgi.

3.4. 4. tests: ne-numeriskas atslēgas

Testētā problēma: pārbauda, vai funkcija spēj sakārtot nevis skaitliskas, bet teksta atslēgas.

Ievade:

```
answer_sheet = {"Q1": "A", "Q2": "B", "Q3": "C"}
student_sheet = {"Q2": "B", "Q1": "B"}
```

Koda fragments:

```
result = dataNormalize(student_sheet, answer_sheet)
assert list(result.keys()) == ["Q1", "Q2", "Q3"]
assert result["Q3"] is None
```

Sagaidāmais rezultāts: atslēgas tiek sakārtotas alfabētiski, trūkstošajai vērtībai tiek piešķirts `None`.

Faktiskais rezultāts: tests izpildīts veiksmīgi.

3.5. 5. tests: pārmērīgas studenta atbildes

Testētā problēma: pārbauda, vai liekās atbildes tiek ignorētas.

Ievade:

```
answer_sheet = {"1": "A", "2": "B"}
student_sheet = {"1": "A", "2": "B", "3": "D", "4": "A", "5": "C"}
```

Koda fragments:

```
result = dataNormalize(student_sheet, answer_sheet)
assert list(result.keys()) == ["1", "2"]
```

Sagaidāmais rezultāts: tiek saglabāti tikai jautājumi, kas atrodas atbilžu lapā.

Faktiskais rezultāts: tests izpildīts veiksmīgi.

3.6. 6. tests: falsy vērtības

Testētā problēma: pārbauda, vai sistēma nezaudē vērtības, kas Python vidē skaitās "falsy", piemēram, "0" vai tukša virkne.

Ievade:

```
answer_sheet = {"1": "0", "2": "", "3": "A"}
student_sheet = {"1": "0", "2": "", "3": "A"}
```

Koda fragments:

```
result = dataNormalize(student_sheet, answer_sheet)
assert result["1"] == "0"
assert result["2"] == ""
```

Sagaidāmais rezultāts: visas ievadītās vērtības tiek saglabātas bez pārveidošanas.

Faktiskais rezultāts: tests izpildīts veiksmīgi.

3.7. 7. tests: visas atbildes nepareizas

Testētā problēma: pārbauda, vai vērtēšanas funkcija korekti piešķir 0 punktus, ja visas atbildes ir nepareizas.

Ievade:

```
mock_load.return_value = {
    "math": {"1": "A", "2": "B", "3": "C"}
}

test = {
    "studentID": "STU001",
    "answers": {
        "math": {"1": "B", "2": "C", "3": "A"}
    }
}
```

Koda fragments:

```
result = GradeTest(test)
assert result["scores"]["math"] == 0
```

Sagaidāmais rezultāts: matemātikas sadaļas rezultāts ir 0.

Faktiskais rezultāts: tests izpildīts veiksmīgi.

3.8. 8. tests: visas atbildes pareizas

Testētā problēma: pārbauda, vai tiek piešķirts maksimālais punktu skaits, ja visas atbildes ir pareizas.

Ievade:

```
mock_load.return_value = {
    "science": {"1": "A", "2": "B"}
}

test = {
    "studentID": "STU002",
    "answers": {
        "science": {"1": "A", "2": "B"}
    }
}
```

Koda fragments:

```
result = GradeTest(test)
assert result["scores"]["science"] == 2
```

Sagaidāmais rezultāts: rezultāts ir 2.

Faktiskais rezultāts: tests izpildīts veiksmīgi.

3.9. 9. tests: trūkstoši jautājumi sadaļā

Testētā problēma: pārbauda, vai vērtēšanas laikā tiek izsaukta normalizācija, ja sadaļā trūkst daļa jautājumu.

Ievade:

```
mock_load.return_value = {
    "section1": {"1": "A", "2": "B", "3": "C"},
    "section2": {"1": "C", "2": "D"}
}

test = {
```

```
    "studentID": "STU003",
    "answers": {
        "section1": {"1": "A", "2": "B", "3": "C"},
        "section2": {"1": "C"}
    }
}
```

Koda fragments:

```
result = GradeTest(test)
assert result["scores"]["section1"] == 3
assert result["scores"]["section2"] == 1
```

Sagaidāmais rezultāts: pirmā sadaļa iegūst 3 punktus, otrā — 1 punktu.

Faktiskais rezultāts: tests izpildīts veiksmīgi.

3.10. 10. tests: minimāla izmēra attēls

Testētā problēma: pārbauda, vai attēla izgriešanas funkcija nesabrūk pie ļoti maza attēla.

Ievade:

```
image = np.ones((50, 50, 3), dtype=np.uint8) * 255
```

Koda fragments:

```
result = extract_document(image)
assert result is not None
assert isinstance(result, np.ndarray)
```

Sagaidāmais rezultāts: funkcija atgriež derīgu attēla objektu.

Faktiskais rezultāts: tests izpildīts veiksmīgi.

3.11. 11. tests: tukšs attēls

Testētā problēma: pārbauda, vai funkcija korekti strādā ar tukšu, baltu attēlu bez objektiem.

Ievade:

```
image = np.ones((400, 400, 3), dtype=np.uint8) * 255
```

Koda fragments:

```
result = get_topic_boxes(image)
assert isinstance(result, list)
```

Sagaidāmais rezultāts: tiek atgriezts saraksts, un funkcija neizsauc kļūdu.

Faktiskais rezultāts: tests izpildīts veiksmīgi.

3.12. 12. tests: viena objekta noteikšana

Testētā problēma: pārbauda, vai tiek atpazīts viens liels taisnstūrveida objekts.

Ievade:

```
image = np.ones((400, 400, 3), dtype=np.uint8) * 255
cv2.rectangle(image, (50, 50), (350, 350), (0, 0, 0), -1)
```

Koda fragments:

```
result = get_topic_boxes(image, debug=False)
assert isinstance(result, list)
```

Sagaidāmais rezultāts: tiek atgriezts saraksts ar atrastajiem objektiem.

Faktiskais rezultāts: tests izpildīts veiksmīgi.

4. Integrācijas testi

4.1. 13. tests: pilns vērtēšanas process

Testētā problēma: pārbauda, vai vairākas funkcijas kopā spēj korekti normalizēt un novērtēt studenta darbu.

Ievade:

```
mock_load.return_value = {
    "section_A": {"1": "A", "2": "B", "3": "C"},
    "section_B": {"1": "D", "2": "A", "3": "B"}
}

test_data = {
    "studentID": "STU12345",
    "answers": {
        "section_A": {"1": "A", "3": "C"},
        "section_B": {"2": "A", "1": "D", "3": "B"}
    }
}
```

Koda fragments:

```
normalized_A = dataNormalize(
    test_data["answers"]["section_A"],
    mock_load.return_value["section_A"]
)
result = GradeTest(test_data)
assert result["scores"]["section_A"] == 2
```

```
assert result["scores"]["section_B"] == 3
```

Sagaidāmais rezultāts: Section A rezultāts ir 2/3, Section B rezultāts ir 3/3.

Faktiskais rezultāts: tests izpildīts veiksmīgi.

4.2. 14. tests: attēlu apstrādes pipeline

Testētā problēma: pārbauda pilnu attēlu apstrādes plūsmu — no sintētiska dokumenta līdz sekciju noteikšanai.

Ievade:

```
image = np.ones((600, 800, 3), dtype=np.uint8) * 240
cv2.rectangle(image, (50, 50), (750, 550), (0, 0, 0), 3)
cv2.rectangle(image, (100, 100), (350, 200), (50, 50, 50), -1)
```

Koda fragments:

```
extracted = extract_document(image, debug=False)
boxes = get_topic_boxes(extracted, debug=False)
assert isinstance(boxes, list)
```

Sagaidāmais rezultāts: dokumenta attēls tiek izgriezts, un tiek atgriezts sekciju saraksts.

Faktiskais rezultāts: tests izpildīts veiksmīgi.

4.3. 15. tests: Flask GET pieprasījums

Testētā problēma: pārbauda, vai `/imgUpload` galapunkts atgriež augšupielādes formu.

Ievade:

```
client = flask_test_app.test_client()
response = client.get("/imgUpload")
```

Koda fragments:

```
assert response.status_code == 200
assert b"<form method=\"POST\" action=\"/imgUpload\"" in response.
    data
```

Sagaidāmais rezultāts: HTTP statuss 200 un HTML forma.

Faktiskais rezultāts: tests izpildīts veiksmīgi.

4.4. 16. tests: Flask POST pieprasījums

Testētā problēma: pārbauda faila augšupielādi, nosaukuma sanitizāciju un faila saglabāšanu.

Ievade:

```
data = {
    "image": (BytesIO(b"fake image data"), "student 01.png")
}
response = client.post("/imgUpload", data=data,
                       content_type="multipart/form-data")
```

Koda fragments:

```
payload = response.get_json()
assert payload["message"] == "Upload successful"
assert payload["filename"] == "student_01.png"
assert (tmp_path / payload["filename"]).exists()
```

Sagaidāmais rezultāts: fails tiek saglabāts, atbilde ir JSON formātā, nosaukums ir sanitizēts.

Faktiskais rezultāts: tests izpildīts veiksmīgi.

5. Testu kopsavilkums

Testu tips	Skaits	Veiksmīgi	Neveiksmīgi
Vienības testi	12	12	0
Integrācijas testi	4	4	0
Kopā	16	16	0

Izpildes laiks: 0.70 sekundes.

6. Secinājumi

Visi 16 testi tika izpildīti veiksmīgi, tādēļ var secināt, ka sistēma ir pietiekami uzticama apskatītajiem robežgadījumiem.